

JMS Report

JMS benchmark

Producer Median Response time	1.333446 ms
Consumer Median Response time	0.014267 ms
Producer Throughput	744.6170804727665 msg/sec
Consumer Throughput	108382.65249807708 msg/sec
Median Latency	1.390217 ms
Average :Latency	1.404647 ms

```
=== JMS BENCHMARK START ===  
=== RESPONSE TIME ===  
SLF4J: No SLF4J providers were found.  
SLF4J: Defaulting to no-operation (NOP) logger implementation  
SLF4J: See https://www.slf4j.org/codes.html#noProviders for further details.  
Producer Median response time: 1.333446 ms  
Consumer Median response time: 0.014267 ms  
=== THROUGHPUT ===  
Messages: 5000  
Producer Throughput: 744.6170804727665 msg/sec  
Consumer Throughput: 108382.65249807708 msg/sec  
=== LATENCY ===  
Median Latency: 1.390217 ms  
Avg Latency: 1.404647 ms  
=== DONE ===
```

Integration

JMS, being a Java EE standard, integrates well within the Java ecosystem but relies heavily on the underlying broker (e.g. ActiveMQ) for broader connectivity. ActiveMQ provides out-of-the-box support for several protocols and systems:

- **Hadoop Ecosystem:** ActiveMQ can integrate with Hadoop via Apache Camel, which provides a JMS component that bridges message queues with HDFS, Hive, and Spark.
- **Databases:** Supports integration with relational databases (MySQL, PostgreSQL) and NoSQL stores (MongoDB, Cassandra) through JDBC connectors or Apache Camel routes.
- **Spring Framework:** First-class support via spring-jms, making it trivial to wire JMS producers/consumers into Spring Boot applications.
- **Apache Camel:** A powerful integration framework with 300+ connectors, making JMS a hub for routing data between virtually any system.
- **REST/HTTP:** ActiveMQ exposes a REST API, allowing non-Java clients to interact with queues.

However, JMS's integrations are largely **manual and code-heavy** compared to modern alternatives. There is no native connector ecosystem like Kafka Connect, meaning every integration requires writing custom bridge code or relying on Camel. It also has **no native support** for stream processing frameworks like Apache Flink or Spark Streaming without an intermediary layer, and connecting to cloud-native data sources (S3, BigQuery, etc.) requires third-party tooling. For data-intensive applications that need to move data across many heterogeneous systems, this is a notable limitation.